Chapter 3 Assignment

Acrobots, Cart-Poles, and Quadrotors

Zihao Liu
Student ID: 220261979
213222647@seu.edu.cn

October 2026

Abstract

This report covers the Chapter 3 assignment. Part I comments on the four textbook examples (Examples 3.1, 3.2, 3.5 and 3.6): the supplied code was run, the numerical results are interpreted using the theory of underactuated robotics, and sensitivity studies on controller weights, initial conditions, actuator limits and physical parameters are reported. Part II solves all exercises of Section 3.8: cart-pole linearisation and balancing, URDF modelling of single/double cart-poles with LQR, controllability of discrete and continuous LTI systems, and the comparison between nonlinear stability and its linearisation. The most important code is reproduced in the report; the complete Python code and notebooks accompany the submission.

Contents

1 Overview and Reproducibility **3**

2 Example 3.1: Dynamics of the Acrobot **4**
2.1 Model and equilibrium 4
2.2 What was run 5
2.3 Linearisation and controllability 5
2.4 LQR design and closed-loop behaviour 5
2.5 Sensitivity study 6

3 Example 3.2: Dynamics of the Cart-Pole System **8**
3.1 Model and linearisation 8
3.2 LQR results and sensitivity 9

4 Example 3.5: LQR for the Acrobot and Cart-Pole **11**
4.1 Theory and design 11
4.2 Comparison of the two systems 11
4.3 Saturation and sensitivity 13

5 Example 3.6: LQR for Quadrotors **13**
5.1 Planar quadrotor 13
5.2 3D quadrotor 15

6 Section 3.8: Exercise 3.1 **17**
6.1 Nonlinear dynamics and linearisation . . . 17
6.2 Linearisation error . . . 17
6.3 LQR and recovery . . . 19

7 Section 3.8: Exercise 3.2 **19**
7.1 URDF construction . . . 19
7.2 Double pendulum . . . 20
7.3 LQR and results . . . 21

8 Section 3.8: Exercise 3.3 **23**
8.1 Discrete double integrator . . . 23
8.2 Continuous LTI controllability . . . 24

9 Section 3.8: Exercise 3.4 **25**
9.1 Model and linearisation . . . 25
9.2 Stability table . . . 27
9.3 Discussion . . . 27

10 Conclusions **27**

11 Code and Reproduction **30**

A Key Code Snippets **31**
A.1 Cart-pole linearisation and LQR . . . 31
A.2 URDF pendulum link and joint . . . 31
A.3 Controllability and two-step reachability . . . 31
A.4 Quadrotor LQR . . . 32

1 Overview and Reproducibility

The assignment consists of two parts:

- **Examples** (code supplied, run and comment on the results): Examples 3.1, 3.2, 3.5 and 3.6.
- **Section 3.8** (all exercises to be solved): Exercises 3.1–3.4.

All simulations use Drake 1.58.0 and the `underactuated` package 2026.9.8 inside the Python environment `/Users/leo/venvs/drake/bin/python` (NumPy 2.5.4, SciPy 1.18.1, Matplotlib 3.11.2). The repository layout is:

Contents	Location
Example notebooks and analyses	`Ch-3/Ch.3-examples/`
Section 3.8 notebooks	`Ch-3/Ch.3 Exercises/`
Section 3.8 analysis scripts	`Ch-3/Ch.3 Exercises/exercises_analysis/`
This report	`Ch-3/Ch3_Assignment_Report/`

The autograder results are:

Notebook	Score
`cartpole_balancing.ipynb`	13/13
`cartpoles_urdf.ipynb`	12/12

The examples are run with the commands listed in Section 11. All random studies use fixed seeds, so the tables are reproducible.

How to read this report

The report contains three layers: (i) *models and equations*, (ii) *numerical experiments*, and (iii) *interpretation*. If a formula looks dense, read the **Intuition** boxes first: they give the physical picture without algebra. The **Recipe** boxes summarise a procedure you can reuse, and the **Common pitfall** boxes point out mistakes that are easy to make.

Part	What is done	What to look for
Example 3.1	Acrobot dynamics, linearisation, LQR	underactuation is not the same as uncontrollability; weak directions; saturation
Example 3.2	Cart-pole dynamics, linearisation, LQR	zero eigenvalues from the cart; right-half-plane zero; slow cart mode
Example 3.5	LQR comparison of acrobot and cart-pole	condition number, non-minimum phase, basin of attraction, actuator limits
Example 3.6	Planar and 3D quadrotor LQR	linearisation at *nonzero* thrust; Q/R trade-off; wind offset
Exercises 3.1–3.4	Section 3.8	linearisation error, URDF frames, reachability proof, stability table

The big picture in five sentences

All systems in this chapter are *underactuated*: there are fewer actuators than degrees of freedom. Linearisation replaces the nonlinear system by its tangent at an equilibrium; LQR stabilises that tangent, so the guarantee is *local*. Two effects limit the usable region: the system may be *non-minimum phase* (it first moves the wrong way) and the actuator may saturate. Controllability answers "can we reach any state?"; it does not answer "how hard is it?". The quadrotor is special only in that the linearisation is taken around a hover point with non-zero thrust.

Symbol	Meaning
$\mathbf{x} = [q, \dot{q}]^\top$	state (positions and velocities)
$\mathbf{x}^\star$	equilibrium / operating point
$A = \partial f/\partial \mathbf{x}$, $B = \partial f/\partial u$	linearisation at $\mathbf{x}^\star$
$\mathcal{C} = [B, AB, \ldots, A^{n-1}B]$	controllability matrix
$Q \succeq 0$, $R \succ 0$	LQR state and input weights
K or D	state-feedback gain ($u = -K(\mathbf{x} - \mathbf{x}^\star)$ or $u = D(\mathbf{x} - \mathbf{x}^\star)$)
RHP zero	right-half-plane zero (non-minimum phase)

2 Example 3.1: Dynamics of the Acrobot

2.1 Model and equilibrium

The acrobot is a two-link planar robot in a vertical plane. The shoulder joint is unactuated and the elbow joint is driven by a torque $u = \tau$. Its manipulator equations are

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \begin{bmatrix} 0 \\ u \end{bmatrix}, \qquad q = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}, \qquad \mathbf{x} = [\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]^\top. \tag{1}$$

The upright equilibrium is $\mathbf{x}^\star = [\pi, 0, 0, 0]^\top$. The default Drake parameters are $m_1 = m_2 = 1$ kg, $l_1 = 1$ m, $l_2 = 2$ m, $l_{c1} = 0.5$ m, $l_{c2} = 1$ m, $I_{c1} = 0.083$ kg m^2, $I_{c2} = 0.33$ kg m^2 and $b_1 = b_2 = 0.1$.

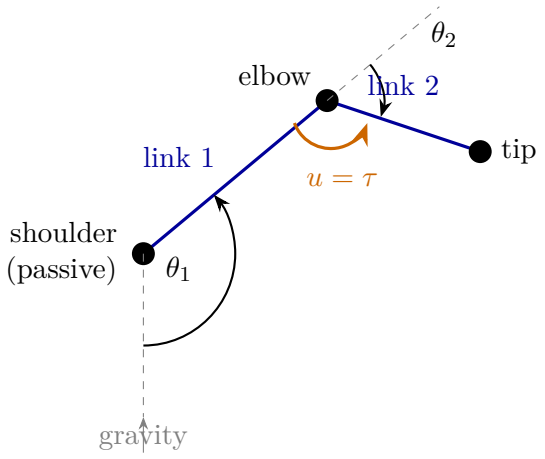


Figure 1: Acrobot schematic. The shoulder is not actuated; the only input is the elbow torque $u = \tau$. θ_1 is measured from the downward vertical and θ_2 from the extension of link 1. Because the shoulder has no motor, the only way to move θ_1 is through the inertial coupling produced by the elbow motion.

Why can a one-motor two-link robot still be controllable?

Imagine holding a chain by one end and moving it with your wrist. The far end is not directly actuated, but it moves because the links are coupled by their mass and inertia. The acrobot works the same way: the elbow torque accelerates link 2, and the reaction on link 1 changes θ_1. This is why the linearised pair (A, B) can have rank 4 even though the shoulder has no motor. The coupling is not equally strong in all directions, however; that is the meaning of "weak controllability" below.

2.2 What was run

The supplied notebook `Exp3'1-acrobot.ipynb` was run. It (i) simulates the response to a constant elbow torque, (ii) linearises the system at $\mathbf{x}^\star$ and checks controllability, and (iii) closes an LQR loop around $\mathbf{x}^\star$. The numerical analysis in `exp1_analysis/exp1_analysis.py` repeats the linearisation and runs 100-trial Monte-Carlo studies with and without the ± 10 N m actuator limit.

2.3 Linearisation and controllability

At $\mathbf{x}^\star$,

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 12.629 & -12.693 & -0.172 & 0.3015 \\ -14.749 & 29.612 & 0.3015 & -0.6033 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ -3.0147 \\ 6.0332 \end{bmatrix}. \tag{2}$$

The open-loop eigenvalues are

$$\lambda(\mathbf{A}) = \{-6.492,\ -2.249,\ +2.231,\ +5.734\}, \tag{3}$$

so the upright position is unstable (two eigenvalues with positive real part). The controllability matrix $\mathcal{C} = [\mathbf{B}, \mathbf{AB}, \mathbf{A}^2\mathbf{B}, \mathbf{A}^3\mathbf{B}]$ has

$$\sigma(\mathcal{C}) = [511.7,\ 123.1,\ 0.0807,\ 0.0715], \qquad \operatorname{rank}\mathcal{C} = 4, \qquad \operatorname{cond}\mathcal{C} \approx 7.15 \times 10^3. \tag{4}$$

Comment. The acrobot is underactuated (one actuator, two degrees of freedom), yet the linearised system is completely controllable. This is the central lesson of Chapter 3: underactuation does not imply loss of controllability; the passive joint is driven through the inertial coupling. The very small last two singular values show, however, that some directions are *weakly* controllable. The slow modes have input participation factors of about 0.048 compared with 3.6 for the fast modes, so driving them requires large inputs or long manoeuvres.

2.4 LQR design and closed-loop behaviour

With $Q = \operatorname{diag}(10, 10, 1, 1)$ and $R = 1$, the solved Riccati equation gives the controller $u = D(\mathbf{x} - \mathbf{x}^\star)$ with

$$D = [\,278.44,\ 112.29,\ 119.72,\ 56.83\,], \tag{5}$$

and closed-loop poles

$$\lambda_{\text{cl}} = \{-10.126,\ -4.238,\ -2.242 \pm 0.039j\}. \tag{6}$$

All poles are in the left half-plane, hence the nonlinear closed loop is locally exponentially stable at $\mathbf{x}^\star$.

The 100-trial Monte-Carlo study (initial state $\mathbf{x}_0 = \mathbf{x}^\star + 0.05\,\mathcal{N}(0, I_4)$, horizon 4 s) gives:

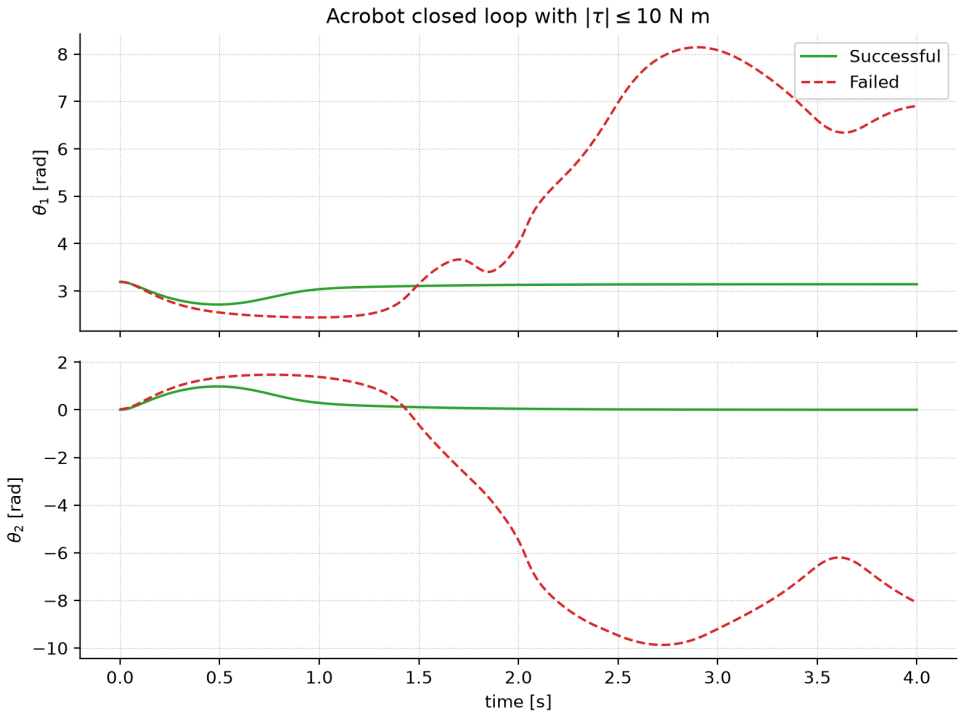


Figure 2: Acrobot: a successful (green) and a failed (red) closed-loop trajectory with the ± 10 N m saturation. The successful trajectory shows the non-minimum-phase undershoot: the elbow first moves far away from zero before returning.

Condition	Success rate	Peak torque (median)
No saturation	98%	11.2 N m
$\lvert\tau\rvert \le 10$ N m	63%	11.2 N m

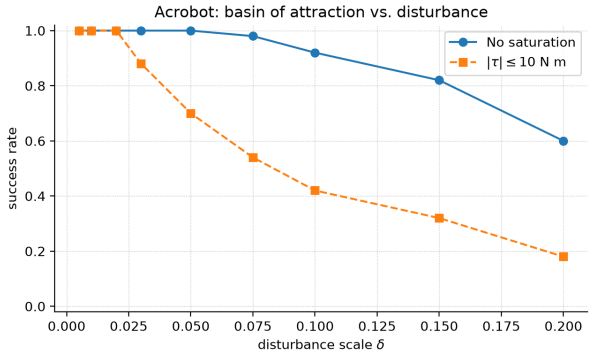
The successful trials have a median peak angular deviation of 0.855 rad and a median settling time of 1.63 s.

Comment. Two effects explain the failures. First, the linear LQR guarantee is local; the nonlinear system has a finite basin of attraction. Second, the saturated actuator turns the problem into a constrained nonlinear one, for which the unconstrained LQR optimality no longer applies. The linearised system is also non-minimum phase: the right-half-plane zeros of $u \to \theta_1$ are ± 2.052 and of $u \to \theta_2$ are $+2.283, -2.304$. State feedback does not move zeros, so the closed loop must first move the state away from the origin before returning; this is visible in Fig. 2.

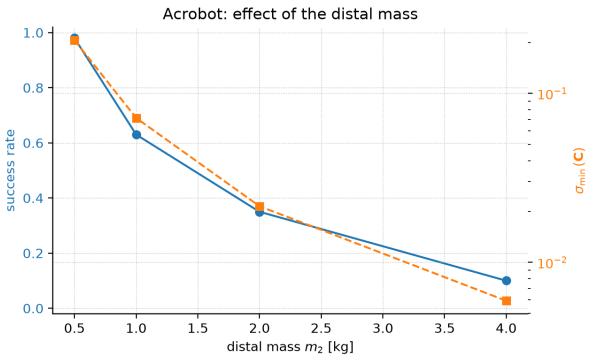
2.5 Sensitivity study

Initial disturbance.

δ	0.005	0.02	0.05	0.075	0.10	0.15	0.20
No saturation	100%	100%	100%	98%	92%	82%	60%
$\lvert\tau\rvert \le 10$ N m	100%	100%	70%	54%	42%	32%	18%



(a) Success rate versus disturbance scale, with and without saturation.



(b) Effect of the distal mass m_2.

Figure 3: Acrobot sensitivity studies. Saturation shrinks the basin of attraction substantially; increasing the distal mass makes the system much harder to control.

Distal mass m_2.

m_2 [kg]	0.5	1.0	2.0	4.0
$\sigma_{\min}(\mathcal{C})$	0.207	0.0715	0.0215	0.0059
$\|D\|$	153.2	328.2	864.3	2655.7
Success rate	98%	63%	35%	10%
Median peak torque [N m]	5.1	11.2	47.8	317.0

Control weight R.

R	0.1	1	10	100
$\|D\|$	577.1	328.2	289.7	285.5
Success rate	65%	63%	62%	62%
Median peak torque [N m]	19.8	11.2	9.8	9.7

Comment. The success rate decreases monotonically with the disturbance size, confirming the finite basin of attraction. The distal mass is the most sensitive physical parameter because it increases the effective inertia and weakens the input leverage; the minimum singular value drops by a factor of 35 and the success rate collapses from 98% to 10%. Increasing R reduces the peak torque only mildly: in the "expensive control" limit the closed-loop poles tend to the mirrors of the stable open-loop poles and the unstable poles, so the gain cannot go to zero for an unstable system.

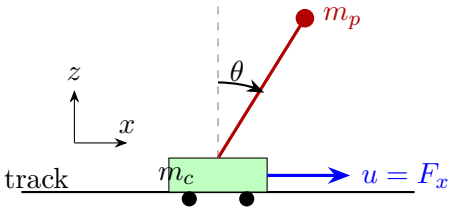


Figure 4: Cart-pole schematic. The cart of mass m_c slides along x; the pole of mass m_p is attached by a passive hinge. The only input is the horizontal force $u = F_x$. θ is measured from the upward vertical ($\theta = \pi$ is upright in the model used in this report).

3 Example 3.2: Dynamics of the Cart-Pole System

3.1 Model and linearisation

What the state and the input mean

x is where the cart is, θ is how far the pole is from upright, $\dot{x}$ and $\dot{\theta}$ are their velocities, and u is the only force we can apply. There is no motor at the pole hinge, so the pole angle can only be changed by moving the cart. This is the whole difficulty of the cart-pole: we must use one actuator to control two coordinates.

The cart-pole has state $\mathbf{x} = [x, \theta, \dot{x}, \dot{\theta}]^\top$ and input $u = F_x$. The model used by the notebook (`underactuated/models/cartpole.urdf`) has $m_c = 10$ kg, $m_p = 1$ kg, $l_c = 0.5$ m and no joint damping. The upright equilibrium is $\mathbf{x}^\star = [0, \pi, 0, 0]^\top$. Linearisation gives

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0.981 & 0 & 0 \\ 0 & 21.582 & 0 & 0 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ 0.1 \\ 0.2 \end{bmatrix}. \tag{7}$$

The open-loop eigenvalues are $0, 0, \pm 4.6456$; the two zeros come from the cart position integrator. The controllability singular values are

$$\sigma(\mathcal{C}) = [4.326,\ 4.326,\ 0.0907,\ 0.0907], \qquad \operatorname{cond} \mathcal{C} \approx 47.7. \tag{8}$$

Comment. The pair is controllable, but the two small singular values correspond to the cart-position direction, which is only weakly controllable. The transfer function from u to x has a right-half-plane zero at $+4.43$: the cart must first move in the opposite direction to bring the pole back. This non-minimum-phase behaviour is visible as a cart undershoot of about 0.3 m for a 0.1 disturbance.

What a right-half-plane zero really means

A zero at $+4.43$ does not mean that the system is unstable. It means that the cart's response to a force starts in the *opposite* direction. Physically: to bring the pole back to the left, the cart first accelerates to the right so that the pole starts falling left; then the cart reverses to catch it. The net initial displacement is therefore opposite to the final one. This "undershoot" is not a controller bug: state feedback cannot move a zero, so LQR keeps it. It also explains why the cart moves about 0.3 m in the wrong direction for a 0.1 disturbance, and why aggressive

saturation can destroy the recovery.

3.2 LQR results and sensitivity

With $Q = \mathrm{diag}(10, 10, 1, 1)$ and $R = 1$, the gain is

$$D = [\,3.1623,\ -254.3925,\ 9.7692,\ -55.1774\,], \qquad u = D(\mathbf{x} - \mathbf{x}^\star), \tag{9}$$

with closed-loop poles

$$-4.7206, \quad -4.5739, \quad -0.3820 \pm 0.3761j. \tag{10}$$

The two fast poles bring the pole back; the slow complex pair is the cart returning to the origin (time constant ≈ 2.6 s). Over a 5 s horizon the cart often has not fully returned even when the pole is balanced.

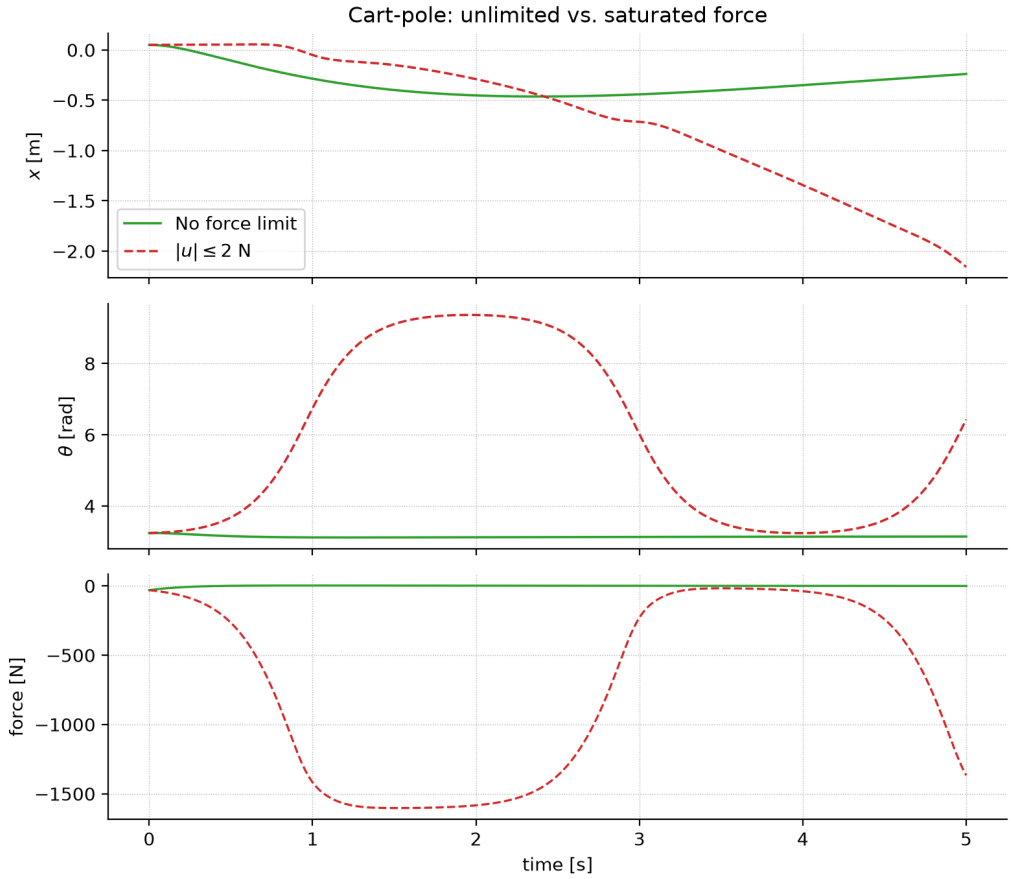
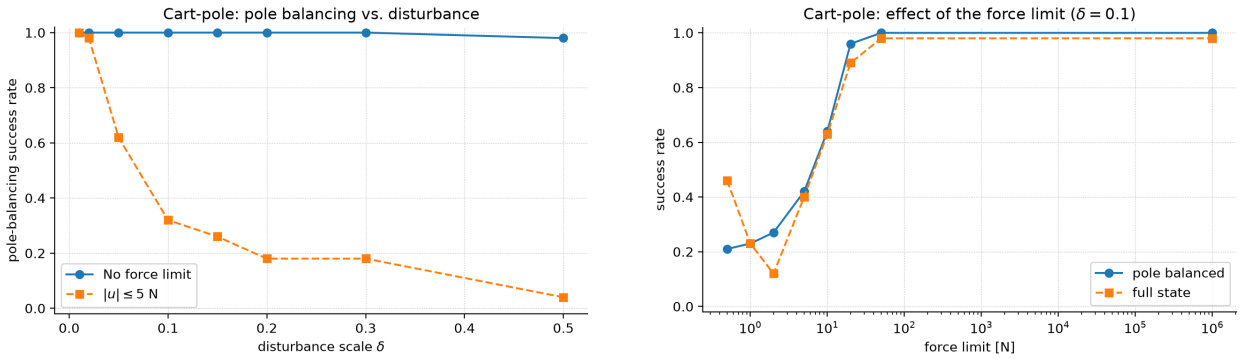


Figure 5: Cart-pole: the same initial condition without a force limit (green) and with a ± 2 N limit (red). The unlimited controller balances the pole, whereas the saturated one loses it.

The 100-trial study with $\mathbf{x}_0 = \mathbf{x}^\star + 0.1\,\mathcal{N}(0, I_4)$ gives:

Condition	Pole balanced	Full state	Peak force (median)
No limit	100%	98%	17.2 N
$\lvert u\rvert \le 20$ N	90%	82%	–
$\lvert u\rvert \le 5$ N	36%	33%	–



(a) Disturbance-size sweep.

(b) Effect of the force limit at $\delta = 0.1$.

Figure 6: Cart-pole sensitivity: the pole itself is robust without saturation, but a realistic force limit quickly reduces the basin of attraction.

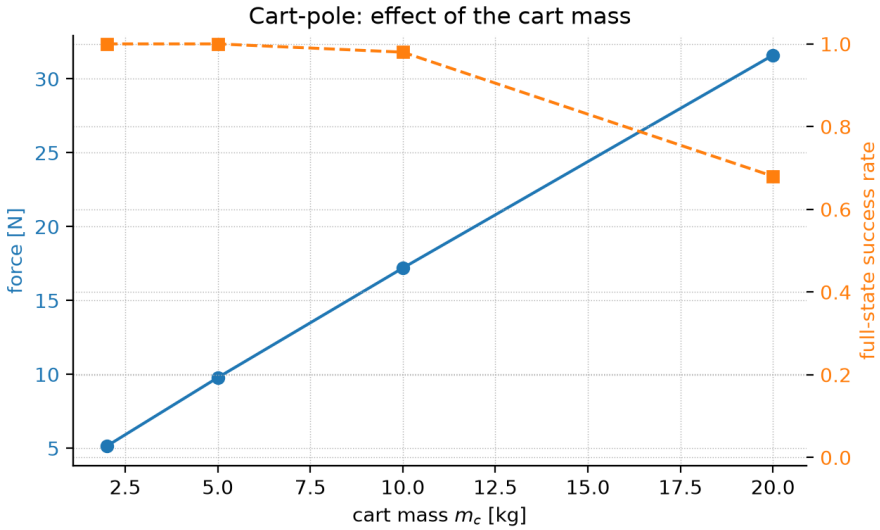


Figure 7: Effect of the cart mass m_c. A heavier cart requires more force and slows down the cart-return mode.

Comment. The original notebook has no force saturation, so the pole almost always balances. The real constraint is the actuator: the median peak force is 17.2 N and the 90th percentile is 40 N. When the limit is reduced to 5 N, saturation destroys the non-minimum-phase undershoot and the pole falls. The cart mass is the dominant physical parameter: from 2 to 20 kg the required force grows from about 5 N to 32 N and the full-state success rate falls from 100% to 68%, while the pole itself remains balanceable without saturation. Increasing R saves force but makes the slow cart mode slower; for $R = 100$ the slow pole is about -0.12, so the cart cannot return within 5 s.

4 Example 3.5: LQR for the Acrobot and Cart-Pole

4.1 Theory and design

For $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u$ and the cost

$$J = \int_0^\infty \left(\mathbf{x}^\top \mathbf{Q} \mathbf{x} + u^\top \mathbf{R} u \right) dt, \tag{11}$$

the optimal state feedback is $u = -\mathbf{R}^{-1}\mathbf{B}^\top P \mathbf{x}$ with P solving the continuous-time algebraic Riccati equation

$$\mathbf{A}^\top P + P\mathbf{A} - P\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top P + \mathbf{Q} = 0. \tag{12}$$

If the linearised pair is stabilisable, LQR returns a stabilising gain for any $\mathbf{Q} \succeq 0$, $\mathbf{R} \succ 0$. The closed-loop eigenvalues determine local exponential stability of the nonlinear system; they say nothing about the size of the basin of attraction or about input saturation.

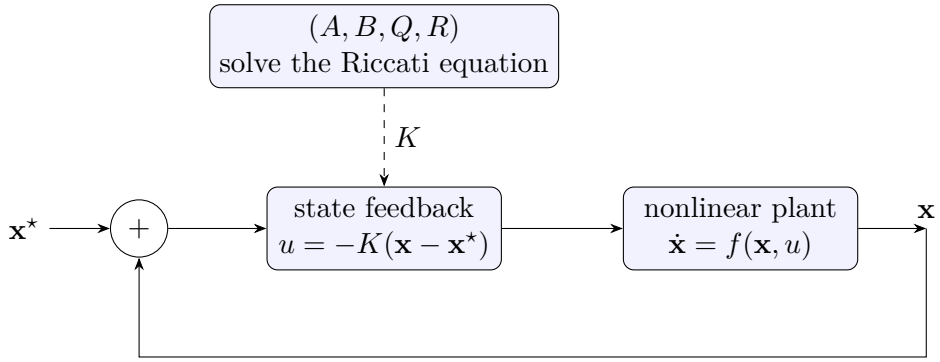


Figure 8: LQR at a glance. The design block solves the algebraic Riccati equation once, offline. The online controller is just a linear state feedback, so it is cheap to run. The nonlinear simulation is what tells us how far from $\mathbf{x}^\star$ the linear guarantee remains useful.

Reading Q and R

Think of the cost as a negotiation between two goals. Q says "keep the state close to the equilibrium"; R says "do not use too much actuator effort". Increasing Q (or decreasing R) makes the controller more aggressive: it corrects faster but uses larger inputs. Increasing R makes it gentler: smaller inputs, slower response, and often a larger steady-state error under a disturbance. The Riccati equation finds the best compromise for the *linear* model; it knows nothing about saturation, friction or the size of the nonlinear basin. That is why the simulations in this report are essential.

4.2 Comparison of the two systems

Comment. Both systems are underactuated and both are controllable. The acrobot is much more weakly controllable (condition number about 150 times larger), because the passive shoulder is driven only through the elbow coupling. The cart-pole has two zero eigenvalues from the cart integrator, which produce a slow closed-loop mode. Both have right-half-plane zeros, so the closed loop necessarily exhibits undershoot.

Table 1: Linearisation and LQR comparison (Drake default parameters).

	Acrobot	Cart-pole
Controllability rank	4	4
Condition number of $\mathcal{C}$	7.15×10^3	47.7
Right-half-plane zeros	$+2.05,\ +2.28$	$+4.43$
Gain D ($u = D(\mathbf{x} - \mathbf{x}^\star)$)	$[278.4, 112.3, 119.7, 56.8]$	$[3.16, -254.4, 9.77, -55.2]$
Closed-loop poles	$-10.13, -4.24, -2.24 \pm 0.04j$	$-4.72, -4.57, -0.38 \pm 0.38j$

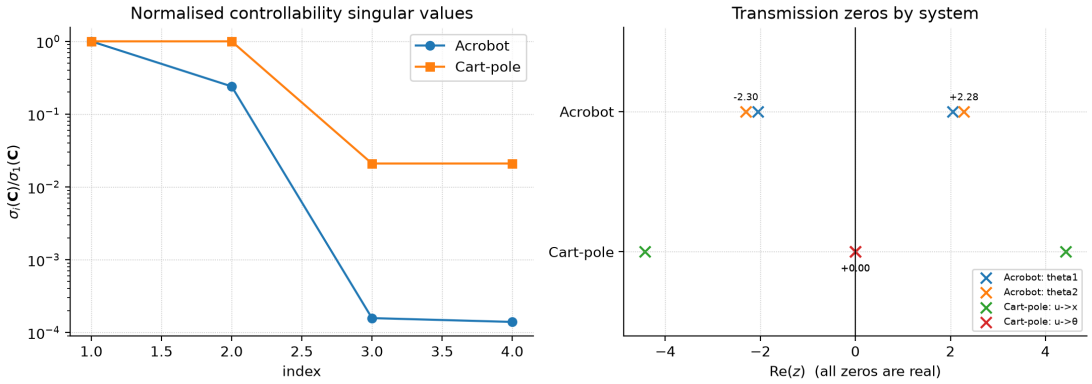


Figure 9: Normalised controllability singular values and right-half-plane zeros. Both systems are controllable, but the acrobot is much more weakly controllable; both are non-minimum phase.

Table 2: Baseline success rates for 100 random disturbances.

System	Input constraint	Success	Peak control
Acrobot	none	98%	median 11.2 N m
Acrobot	$\lvert\tau\rvert \le 10$ N m	63%	–
Cart-pole	none	100% (pole), 99% (full)	median 18.4 N
Cart-pole	$\lvert u\rvert \le 20$ N	90% (pole), 82% (full)	–
Cart-pole	$\lvert u\rvert \le 5$ N	36% (pole), 33% (full)	–

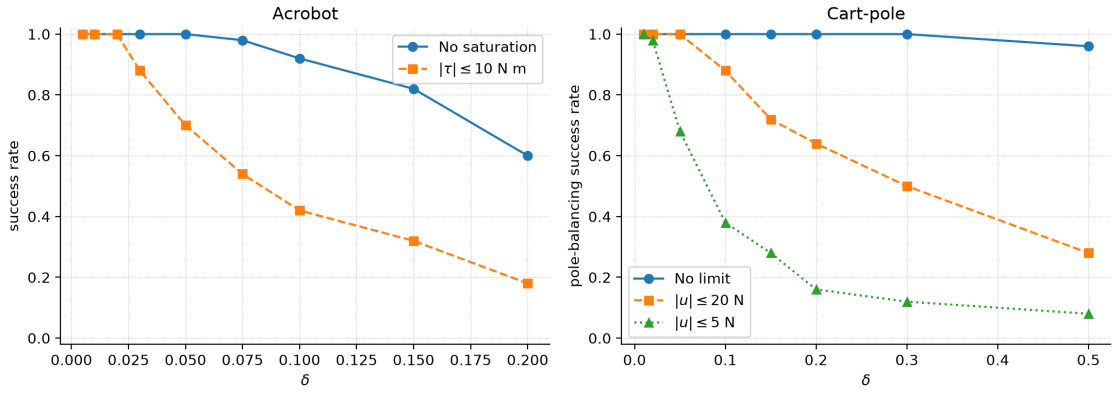
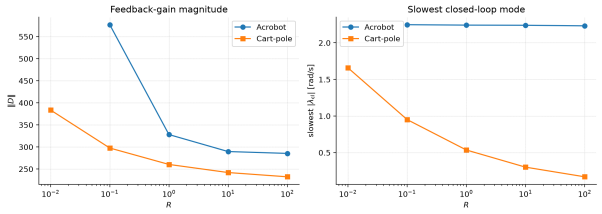
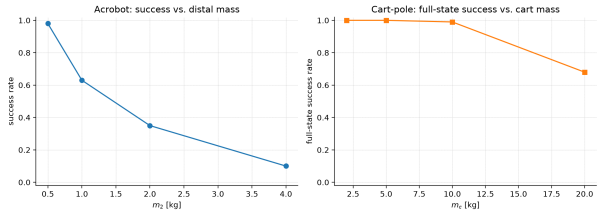


Figure 10: Success rate versus disturbance scale for the acrobot with and without torque saturation (left) and the cart-pole with different force limits (right).



(a) Effect of R on the gain and the slow pole.



(b) Physical-parameter sensitivity.

Figure 11: Weight and parameter sensitivity. For the cart-pole, increasing R slows the cart-return mode; for the acrobot, the distal mass is the critical parameter.

4.3 Saturation and sensitivity

Comment. The acrobot needs a finite minimum torque even in the "expensive control" limit, because its linearisation has unstable poles. The cart-pole's slow pole moves from 1.66 to 0.17 rad/s as R grows from 0.01 to 100, so the full-state success rate falls from 100% to 27% even though the pole remains balanced. The physical-parameter studies show two different failure modes: the acrobot becomes genuinely uncontrollable in practice when m_2 is large, whereas the cart-pole mainly needs more force and a longer time for the cart to return.

5 Example 3.6: LQR for Quadrotors

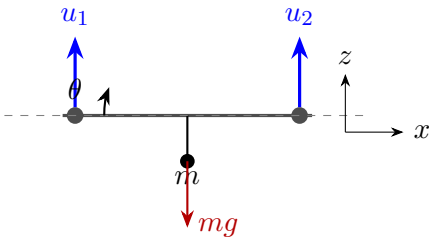


Figure 12: Planar quadrotor schematic. The two propeller thrusts are the only inputs. At hover, $u_1 + u_2 = mg$ and $\theta = 0$. The vehicle can translate by tilting and then using the horizontal component of the total thrust.

5.1 Planar quadrotor

The planar quadrotor has state

$$\mathbf{x} = [x, z, \theta, \dot{x}, \dot{z}, \dot{\theta}]^\top \tag{13}$$

and inputs $u = [u_1, u_2]^\top$ (two propeller thrusts). With mass m, arm length l, inertia I and gravity g, the dynamics are

$$\ddot{x} = \frac{1}{m}\left(w_x - (u_1 + u_2)\sin\theta\right), \tag{14}$$

$$\ddot{z} = \frac{1}{m}\left(w_y + (u_1 + u_2)\cos\theta\right) - g, \tag{15}$$

$$\ddot{\theta} = \frac{l}{I}(u_1 - u_2), \tag{16}$$

where w_x, w_y denote a disturbance (wind) force. The model parameters are $m = 0.486$ kg, $l = 0.25$ m and $I = 0.00383$ kg m^2. The hover equilibrium is $\theta = 0$, $u_1 = u_2 = mg/2$. The supplied planar notebook `Exp3'6-planar-quadrotor.ipynb` was run and the quantitative studies below were produced by `quadrotor_analysis.py`. The example uses

$$\mathbf{Q} = \mathrm{diag}\left(10, 10, 10, 1, 1, \frac{l}{2\pi}\right), \qquad \mathbf{R} = \begin{bmatrix} 0.1 & 0.05 \\ 0.05 & 0.1 \end{bmatrix}. \tag{17}$$

Why LQR "works out of the box" for a quadrotor

The 3D quadrotor has four propeller inputs but six degrees of freedom, so it is underactuated: it cannot independently command position and attitude at the same instant. Nevertheless, the hover equilibrium is *stabilisable*. The linearisation is taken around a point where the total thrust balances gravity ($u_i = mg/4$), not around zero thrust; this is the key detail. The LQR then coordinates the four thrusts to correct roll/pitch, which produces horizontal acceleration, and finally brings the vehicle back to the hover position. There is no integral action, so a constant wind causes a steady-state position offset; this is visible in the sensitivity study.

Baseline. The resulting LQR has closed-loop poles

$$-80.75,\ -15.86,\ -6.59,\ -3.60,\ -2.72 \pm 1.61j. \tag{18}$$

From the initial condition $(0, 0, 0.2, 0, 0, 0)$ the controller settles in about 0.06 s with a peak thrust of 5.17 N. Ten random initial conditions with standard deviation 0.1 all recover, i.e. a 100% success rate.

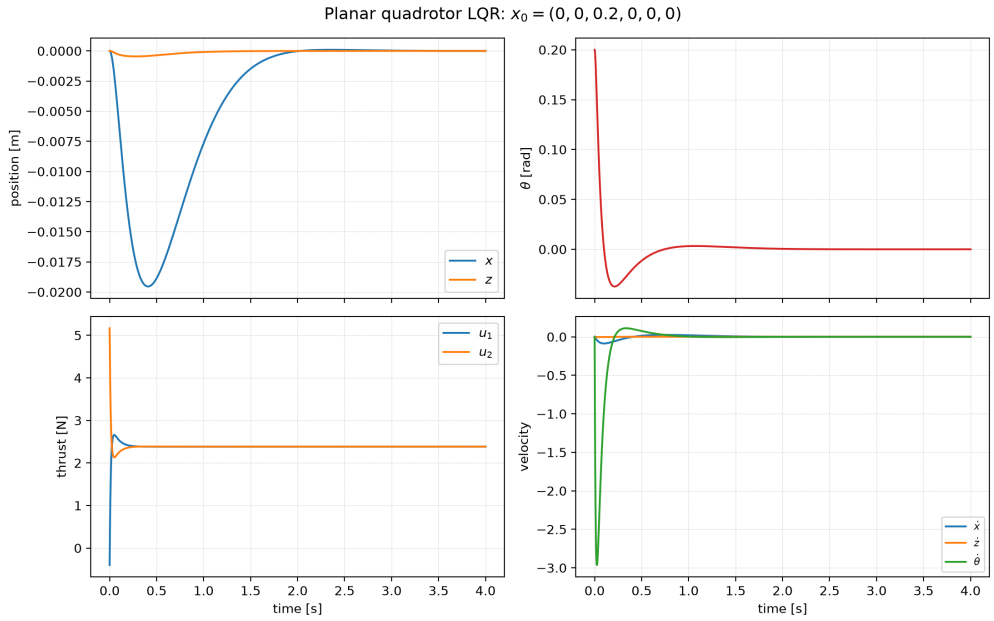


Figure 13: Planar quadrotor LQR from $(0, 0, 0.2, 0, 0, 0)$: position, attitude, velocities and the two thrusts. The initial tilt produces a downward excursion before the vehicle returns to hover.

Sensitivity. Four studies are shown in Fig. 14:

1. increasing the position/attitude weight in $\mathbf{Q}$ makes the response faster but increases the peak thrust;
2. increasing $\mathbf{R}$ makes the controller less aggressive: the peak thrust decreases while the settling time increases;
3. a constant horizontal wind produces a steady-state position offset, because LQR has no integral action: the offset grows from 0.073 m at 0.25 N to 0.554 m at 2 N;
4. the initial tilt is recovered for all tested values up to 0.7 rad, with the settling time and thrust increasing with the tilt.

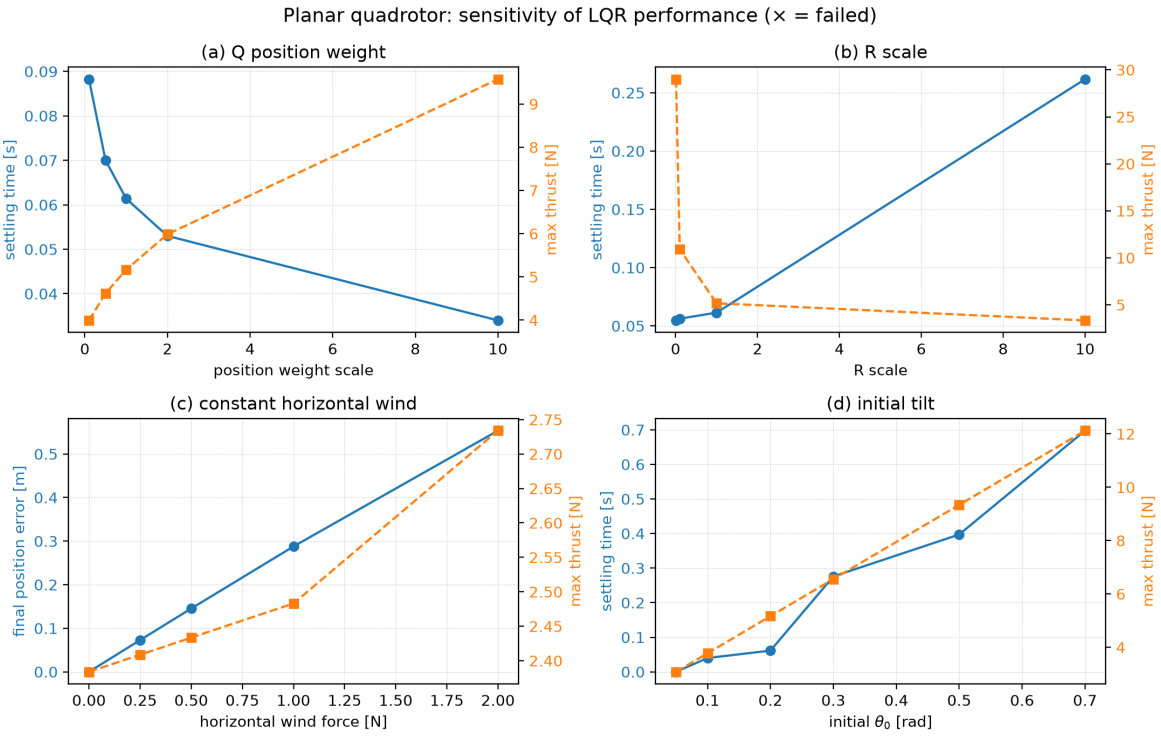


Figure 14: Planar quadrotor sensitivity: (a) $\mathbf{Q}$ position weight, (b) $\mathbf{R}$ scale, (c) constant horizontal wind, (d) initial tilt. The crosses denote failed runs; in (c) the nonzero steady-state offset is the expected consequence of no integral action.

5.2 3D quadrotor

For the 3D model the state is

$$\mathbf{x} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, p, q, r]^\top \tag{19}$$

with four propeller forces $u_1, \ldots, u_4$. The Drake model has mass $m = 0.775$ kg and inertia $\mathrm{diag}(0.0015, 0.0025, 0.0035)$ kg m^2. Hover at $(0, 0, 1)$ requires $u_i = mg/4$. The closed-loop poles of

the LQR below are -141.4, -84.8, -13.6, -3.25, -3.16, -3.16, $-2.40 \pm 1.55j$ and $-2.21 \pm 2.22j$ (each complex pair twice). Following the textbook notebook, we use

$$\mathbf{Q} = \mathrm{diag}(10, \ldots, 10, 1, \ldots, 1), \qquad \mathbf{R} = I_4. \tag{20}$$

The official `StabilizingLQRController` and our own LQR give the same baseline performance from $(0, 0, 1, 0.2, 0.1, 0, 0, \ldots, 0)$: final position error $< 10^{-5}$ m, peak attitude deviation 0.2 rad, settling time 1.08 s and peak thrust 3.0 N.

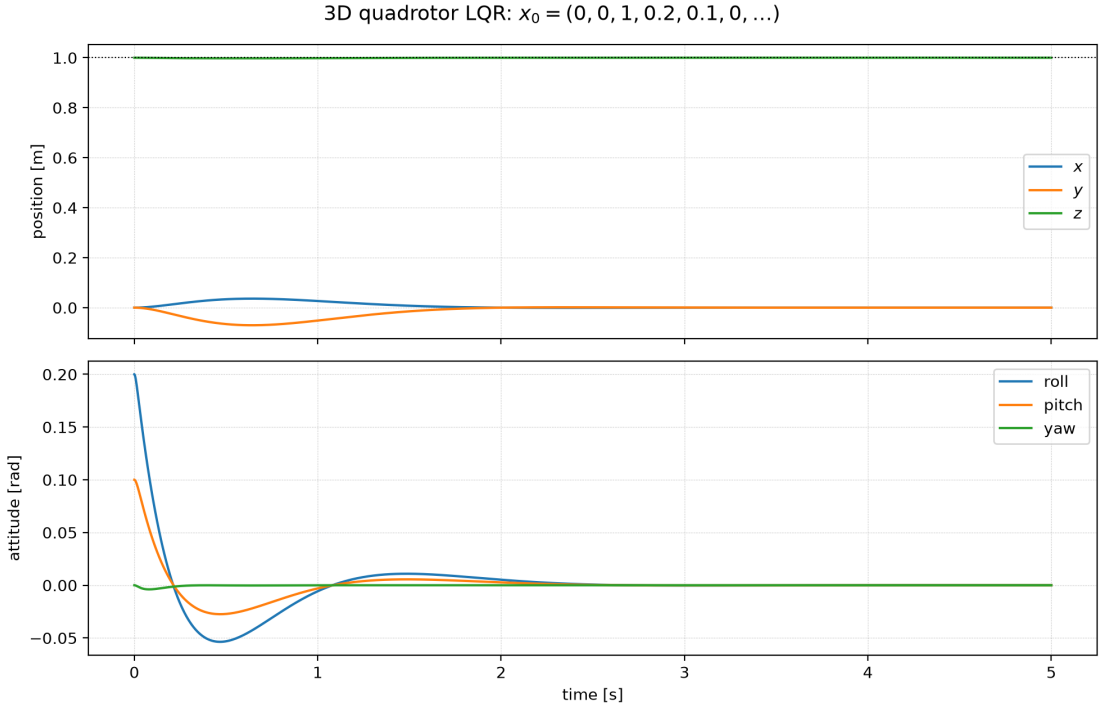


Figure 15: 3D quadrotor LQR from a 0.2 rad roll and 0.1 rad pitch perturbation. Position (top) and attitude (bottom).

Sensitivity. Figure 16 reports the settling time and peak thrust for variations of $\mathbf{Q}$, $\mathbf{R}$ and the initial tilt:

- increasing the position/attitude weight reduces the settling time from 2.02 s to 0.08 s but raises the peak thrust from 2.68 N to 3.99 N;
- increasing $\mathbf{R}$ reduces the peak thrust from 12.7 N to 2.26 N with only a small increase in settling time;
- larger initial tilts require more thrust and take longer to settle (1.44 s at 0.5 rad), but all tested cases recover.

Comment. LQR works out of the box for the quadrotor because the linearisation is taken around a *non-zero* thrust that balances gravity. The vehicle is underactuated (four inputs for six degrees of freedom), but the hover equilibrium is stabilisable. The $\mathbf{Q}/\mathbf{R}$ trade-off is the usual one: aggressive attitude/position weighting gives fast response at the price of thrust; large $\mathbf{R}$ is gentle but slow. Since LQR has no integral action, a constant wind produces a steady-state position error; rejecting it would require integral action, a disturbance observer or MPC.

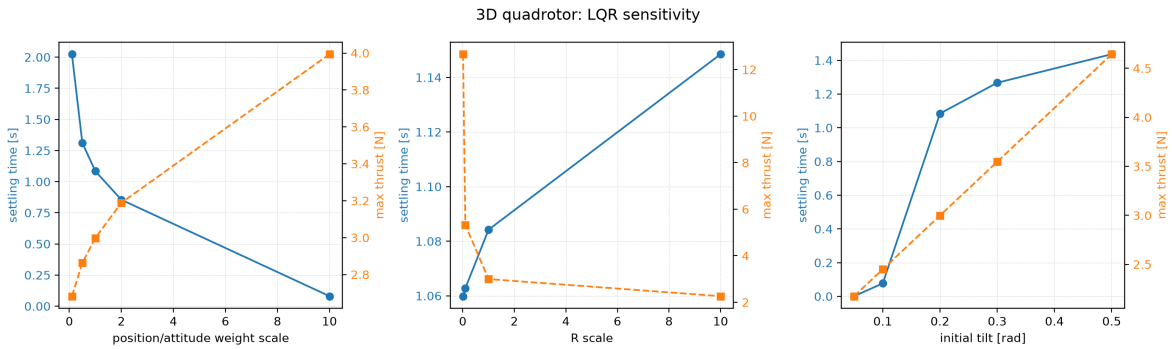


Figure 16: 3D quadrotor sensitivity: settling time (left axis) and peak thrust (right axis) vs. **Q** weight, **R** scale and initial tilt.

6 Section 3.8: Exercise 3.1

6.1 Nonlinear dynamics and linearisation

For the cart-pole with $m_c = m_p = 1$, $l = 1$, $g = 9.81$ and $\mathbf{x} = [x, \theta, \dot{x}, \dot{\theta}]^\top$, the equations used in the notebook are

$$\ddot{x} = \frac{u + \sin\theta\left(\dot{\theta}^2 + g\cos\theta\right)}{1 + \sin^2\theta}, \tag{21}$$

$$\ddot{\theta} = \frac{-u\cos\theta - \dot{\theta}^2\cos\theta\sin\theta - 2g\sin\theta}{1 + \sin^2\theta}. \tag{22}$$

At the upright equilibrium $\mathbf{x}^\star = [0, \pi, 0, 0]^\top$,

$$\mathbf{A}_{\text{lin}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 9.81 & 0 & 0 \\ 0 & 19.62 & 0 & 0 \end{bmatrix}, \qquad \mathbf{B}_{\text{lin}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}. \tag{23}$$

Do not use a linear controller everywhere

A linear controller is designed for the tangent at one point. If the state is far from that point, the real dynamics can differ so much that the controller no longer helps. In this exercise the controller works from $\theta = 0.99\pi$, 0.90π and 0.85π, but fails from $\theta = \pi/2$ and from the downward equilibrium $\theta = 0$. The failure is not numerical: the linear model is simply the wrong model there. This is also why a real cart-pole needs a swing-up controller before the LQR can take over.

6.2 Linearisation error

Comments. The linear model is not valid at all points: at $\theta = \pi/2$ the error is 2.66 times the field magnitude, and at $\theta = 0$ the error is 68.9. When $\theta \neq \pi$, the error depends on u because the nonlinear input gains are $u/(1 + \sin^2\theta)$ and $-u\cos\theta/(1 + \sin^2\theta)$, whereas the linear model uses the constants $(1, 1)$. The cart position x does not appear in f, so it does not affect the error. This is why case 1 has a larger error than case 2 even though its angle is closer to π: it has a large input.

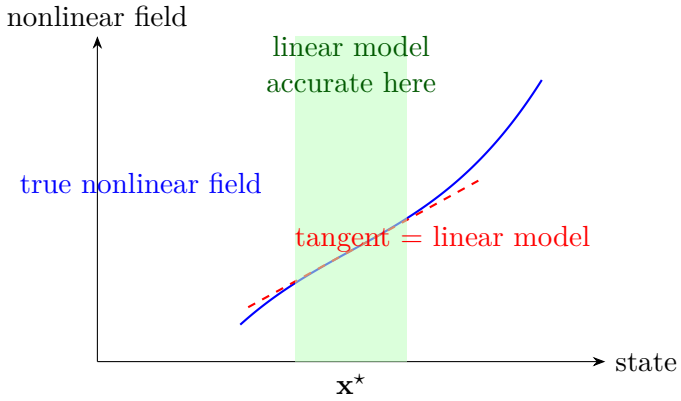


Figure 17: Why linearisation is local. The red dashed line is the tangent (linear model) at $\mathbf{x}^\star$; the blue curve is the true nonlinear field. They agree near $\mathbf{x}^\star$ (green band) and diverge farther away. The six test points in the table below sample different distances from the equilibrium.

Table 3: Linearisation error for the six conditions of the notebook.

Case	$\mathbf{x}$	u	$e(\mathbf{x}, u)$	$\|f(\mathbf{x}, u)\|$
0	$[0, 0.99\pi, 0, 0]$	0	0.0009	0.688
1	$[0, 0.90\pi, 0, 0]$	-10	2.354	18.450
2	$[0, 0.85\pi, 0, 0]$	0	2.289	8.085
3	$[0, 0.50\pi, 0, 0]$	0	26.054	9.810
4	$[0, 0, 0, 0]$	0	68.913	0
5	$[1, \pi, 0, 0]$	10	≈ 0	14.142

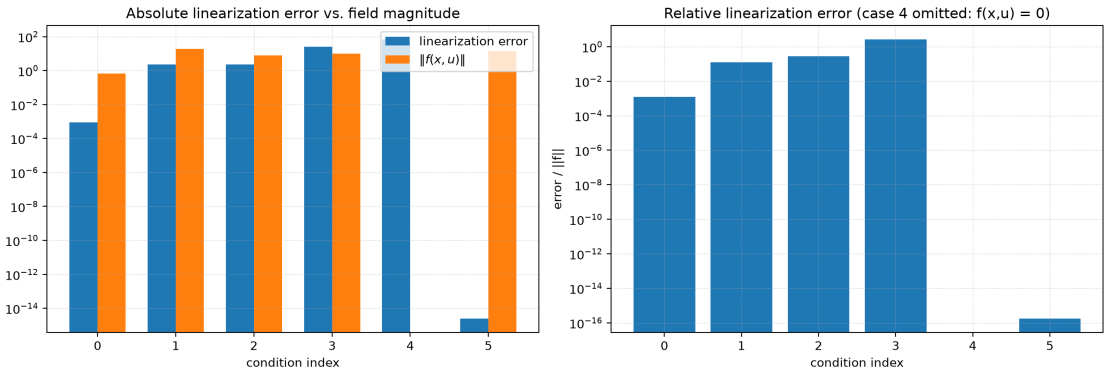


Figure 18: Absolute and relative linearisation errors. The linear model is accurate only near the upright equilibrium.

6.3 LQR and recovery

With $\mathbf{Q} = I_4$, $\mathbf{R} = 1$ the gain is

$$K = [\,-1.0000,\ 51.0612,\ -2.7221,\ 12.8277\,], \qquad u = -K(\mathbf{x} - \mathbf{x}^\star), \tag{24}$$

and the closed-loop poles are

$$-5.0417, \quad -3.9438, \quad -0.5600 \pm 0.4239j. \tag{25}$$

A 5 s nonlinear simulation from the six initial states recovers from

$$\boxed{\texttt{system_recovers_from_states} = [0, 1, 2, 5]}. \tag{26}$$

Cases 0–2 are close to the upright point; case 5 has $\theta = \pi$ and only a cart offset, which is linearly controllable. Case 3 ($\theta = \pi/2$) and case 4 ($\theta = 0$) are outside the linearisation region and the controller fails, as predicted by the error table.

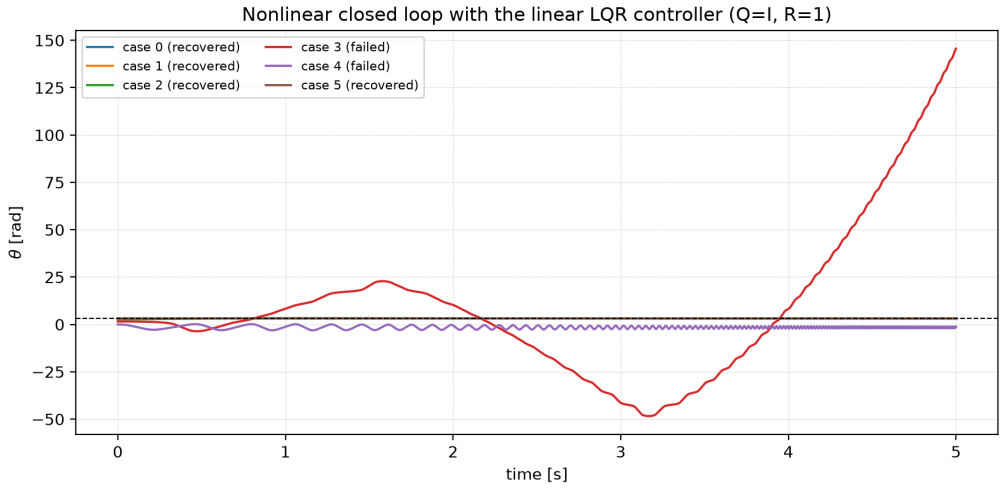


Figure 19: Closed-loop angle histories for the six initial states. Only cases 0, 1, 2 and 5 converge to the upright position.

7 Section 3.8: Exercise 3.2

7.1 URDF construction

The single cart-pole is built from three XML blocks:

- **links**: a base with mass 1 at its origin, a pendulum link whose ball (mass 1) is at $z = -1$, a spherical visual of radius 0.05 and a cylindrical rod of length 1 and radius 0.01 centred at $z = -0.5$;
- **joints**: a prismatic joint x from the world to the base, and a continuous joint θ_0 from the base to the pendulum, with origin $(0, 0, 0)$ and axis $(0, -1, 0)$;
- **transmission**: a simple transmission that maps the actuator force to the prismatic joint.

The unstable equilibrium is $\mathbf{x}^\star = [0, \pi, 0, 0]^\top$.

7.2 Double pendulum

The second joint is attached at the ball of the first pendulum: `origin xyz="0 0 -1"` in the `pendulum0` frame. The second link has its ball at $z = +1$ in its own frame. Hence, for $\theta_1 = 0$ the second pendulum points back towards the cart and exactly overlaps the first; for $\theta_0 = \theta_1 = \pi$ both pendula stand upright one above the other. The unstable equilibrium is

$$\mathbf{x}^\star = [0, \pi, \pi, 0, 0, 0]^\top. \tag{27}$$

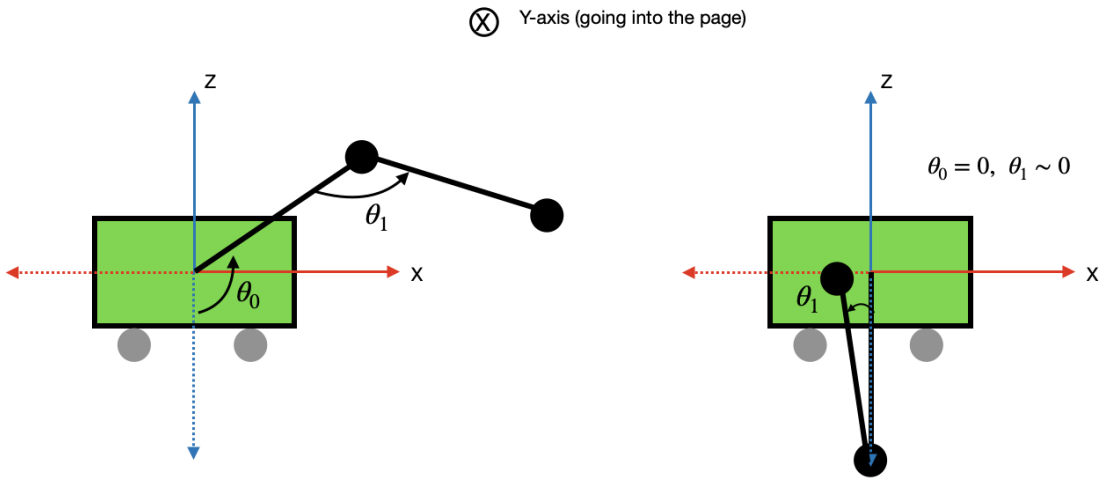


Figure 20: Geometry of the double pendulum (textbook figure). Left: general configuration, with θ_0 measured from the downward vertical and θ_1 measured relative to the first pendulum. Right: the home configuration $\theta_0 = \theta_1 = 0$. The second pendulum points back towards the cart and overlaps the first; this is exactly what the notebook asks you to observe.

Reading URDF frames (the part that is easy to get wrong)

A URDF is a chain of *frames*, not a chain of absolute coordinates.

1. The `<origin>` of a joint is written in the *parent link* frame. Here `theta1` has origin $(0, 0, -1)$ in `pendulum0`, so the joint sits at the ball of pendulum 0.
2. The child link frame is the joint frame. Therefore the inertial and visual origins of `pendulum1` are written in the `pendulum1` frame, i.e. relative to that joint.
3. In the home configuration $\theta_1 = 0$, the child frame is not rotated. Putting the ball at $z = +1$ in this frame makes it go from the joint at $z = -1$ back up to $z = 0$: the two rods overlap.
4. At $\theta_0 = \theta_1 = \pi$, pendulum 0 points up (ball at $z = +1$); the second joint is at that ball, and $\theta_1 = \pi$ turns the second rod so its ball is at $z = +2$. Both links stand upright, one above the other.

Listing 1: Key part of the double-pendulum URDF.

```
<link name="pendulum1">
  <inertial>
```

```
    <origin xyz="0 0 1" rpy="0 0 0" />
    <mass value="1" />
    <inertia ixx="0" ixy="0" ixz="0" iyy="0" iyz="0" izz="0" />
  </inertial>
  <visual>
    <origin xyz="0 0 1" />
    <geometry><sphere radius="0.05" /></geometry>
  </visual>
  <visual>
    <origin xyz="0 0 0.5" />
    <geometry><cylinder length="1" radius="0.01" /></geometry>
  </visual>
</link>
<joint name="theta1" type="continuous">
  <parent link="pendulum0" />
  <child link="pendulum1" />
  <origin xyz="0 0 -1" />
  <axis xyz="0 -1 0" />
</joint>
```

7.3 LQR and results

The LQR is synthesised directly from the nonlinear `MultibodyPlant` at $\mathbf{x}^\star$:

Listing 2: Wiring the LQR controller.

```
context = cartpole.CreateDefaultContext()
context.get_mutable_continuous_state_vector().SetFromVector(x_star)
cartpole.get_actuation_input_port().FixValue(context, [0])
input_i = cartpole.get_actuation_input_port().get_index()
lqr = LinearQuadraticRegulator(
    cartpole, context, Q, R, input_port_index=input_i)
lqr = builder.AddSystem(lqr)
builder.Connect(cartpole.get_state_output_port(), lqr.get_input_port(0))
builder.Connect(lqr.get_output_port(0), cartpole.get_actuation_input_port())
```

For the single pendulum with $\mathbf{Q} = \mathrm{diag}(10, 10, 1, 1)$, $\mathbf{R} = 1$:

$$K = [\,-3.1623,\ 61.3987,\ -5.2854,\ 16.1699\,], \tag{28}$$

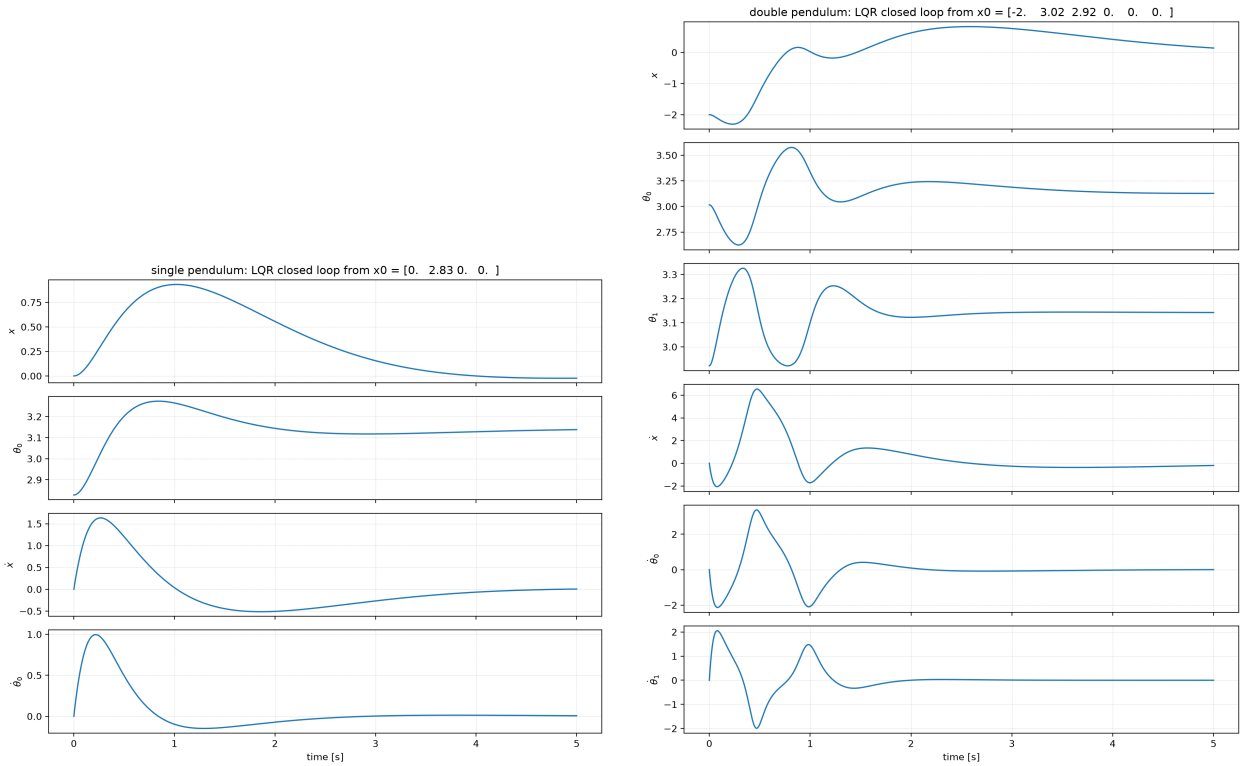
closed-loop poles $-4.8948, -4.0983, -0.9457 \pm 0.8075j$; after 5 s the angle error is 0.0036 rad and the peak force is 19.3 N. For the double pendulum with $\mathbf{Q} = \mathrm{diag}(10, 10, 10, 1, 1, 1)$ the gain is

$$K = [\,3.1623,\ -54.8416,\ -300.4370,\ 7.4384,\ -54.0081,\ -68.9418\,], \tag{29}$$

with closed-loop poles $-7.5798, -6.1385, -3.6749, -3.4281, -0.7754 \pm 0.6611j$. After 5 s the two angle errors are 0.0142 rad and 0.0007 rad; the peak force is 66.6 N. The cart is still slowly returning to the origin, consistent with the slow pair of poles.

The autograder gives 12/12:

```
Total score is 12/12.
Test x_star single pendulum        1/1
Test x_star double pendulum        1/1
Test single-pendulum dynamics      3/3
```



(a) Single pendulum.

(b) Double pendulum.

Figure 21: LQR closed-loop simulations for the URDF models.

```
Test double-pendulum dynamics      3/3
Test single-pendulum LQR states    2/2
Test double-pendulum LQR states    2/2
```

8 Section 3.8: Exercise 3.3

8.1 Discrete double integrator

The dynamics are

$$x_1[n+1] = x_1[n] + x_2[n], \qquad x_2[n+1] = x_2[n] + u[n], \qquad x_1, x_2, u \in \mathbb{Z}, \tag{30}$$

so a transition from $s_{i,j}$ is

$$s_{i,j} \longmapsto (i + j,\ j + u). \tag{31}$$

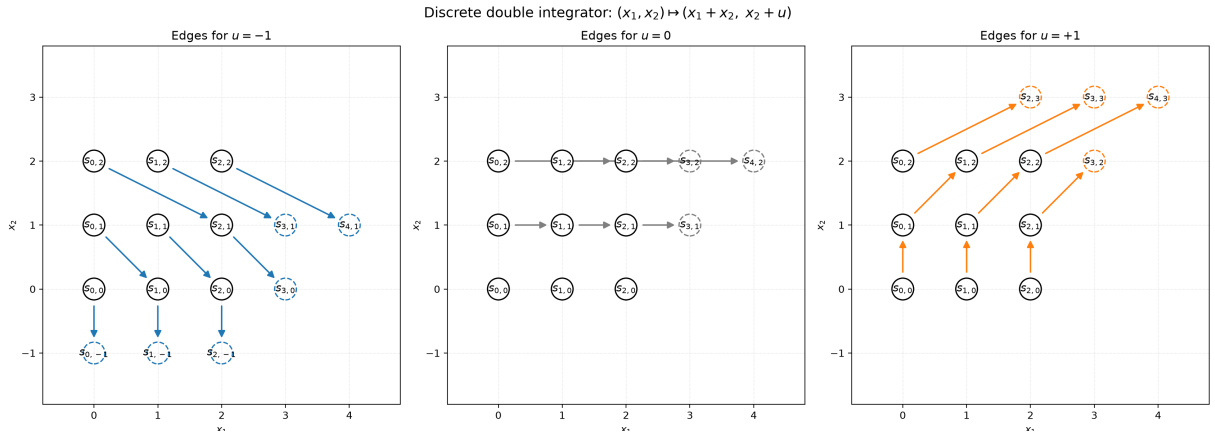


Figure 22: Edges of the graph for $u = -1, 0, +1$. The nine source nodes are drawn as solid circles; destinations outside the truncated grid are drawn as dashed circles. For $u = 0$ the three bottom nodes have self-loops.

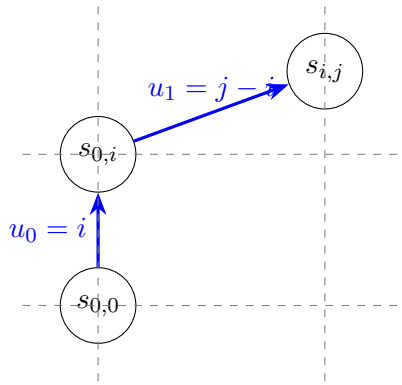


Figure 23: Two-step construction from $s_{0,0}$ to $s_{i,j}$. The first step only changes x_2 (we move vertically in the grid); the second step uses that x_2 to move x_1 (diagonally). One step cannot reach $i > 0$ because $x_1[1] = 0$ regardless of u_0.

How to think about the discrete map

Write the state after N steps by unrolling the recurrence:

$$x_1[N] = \sum_{k=0}^{N-2} (N-1-k)u_k, \qquad x_2[N] = \sum_{k=0}^{N-1} u_k.$$

For $N = 1$ the first sum is empty, so x_1 cannot change; for $N = 2$ we have two control variables and two equations, which is exactly enough to hit any integer pair. This is a controllability statement in disguise: the number of independent controls (2) matches the number of states (2).

Minimal number of steps from $s_{0,0}$. After one step the state is $(0, u_0)$, so $x_1 = 0$ and no $s_{i,j}$ with $i > 0$ is reachable. After two steps the state is $(u_0, u_0 + u_1)$; choosing

$$u_0 = i, \qquad u_1 = j - i \tag{32}$$

reaches any $s_{i,j}$, in particular the required $j > i > 0$. Hence the minimum is $\boxed{2}$.

Arbitrary initial and final states. From s_{i_1,j_1}, two steps give

$$x_1[2] = i_1 + 2j_1 + u_0, \qquad x_2[2] = j_1 + u_0 + u_1. \tag{33}$$

The choice

$$u_0 = i_2 - i_1 - 2j_1, \qquad u_1 = j_2 + j_1 + i_1 - i_2 \tag{34}$$

reaches s_{i_2,j_2} in two steps. One step is not enough in general because it gives $x_1[1] = i_1 + j_1$, independent of u_0; so the minimum guaranteed number is again $\boxed{2}$. (For a particular target with $i_2 = i_1 + j_1$, one step may suffice.)

8.2 Continuous LTI controllability

For $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u$, the controllability matrix is $\mathcal{C} = [\mathbf{B}, \mathbf{AB}, \ldots, \mathbf{A}^{n-1}\mathbf{B}]$.

Table 4: Controllability of the four LTI pairs.

Pair	$\mathcal{C}$	rank	Controllable?	Reason
$(\mathbf{A}_1, \mathbf{B}_1)$	$\begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$	1	No	x_1 decoupled
$(\mathbf{A}_2, \mathbf{B}_2)$	$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$	1	No	$\dot{z} = z$ for $z = x_1 - x_2$
$(\mathbf{A}_3, \mathbf{B}_3)$	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	2	Yes	double integrator
$(\mathbf{A}_4, \mathbf{B}_4)$	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 2 & 0 \end{bmatrix}$	4	Yes	cart-pole linearisation

For $(\mathbf{A}_2, \mathbf{B}_2)$, $z = x_1 - x_2$ satisfies $\dot{z} = z$, which is independent of u (it is not conserved; it grows like e^t). Equivalently, $\mathbf{B}_2$ is orthogonal to the left eigenvector $[1, -1]^\top$. For $(\mathbf{A}_4, \mathbf{B}_4)$, the eigenvalues are $0, 0, \pm\sqrt{2}$ and the PBH test $\operatorname{rank}[\mathbf{A}_4 - \lambda I, \mathbf{B}_4] = 4$ holds for each eigenvalue.

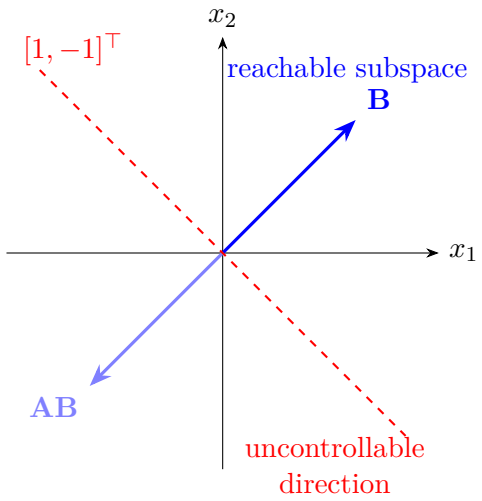


Figure 24: Geometry of the (A_2, B_2) example. Both $B_2 = [1,1]^\top$ and $A_2B_2 = B_2$ point along the diagonal, so only the diagonal is reachable. The orthogonal direction $[1,-1]^\top$ is never affected by the input. This is why the controllability matrix has rank 1 even though the state space is two-dimensional.

Testing controllability without guessing

1. Build $\mathcal{C} = [\mathbf{B}, \mathbf{AB}, \ldots, \mathbf{A}^{n-1}\mathbf{B}]$ and compute its rank. Full rank means every state is reachable.
2. If the rank is deficient, the left null space of $\mathcal{C}$ gives the unreachable directions (equivalently, use the PBH test $\text{rank}[\mathbf{A} - \lambda I, \mathbf{B}] = n$ at each eigenvalue).
3. Rank is binary; the singular values tell you how *hard* each direction is to drive. A full-rank system can still be very weakly controllable.
4. Underactuation is a different question: compare the number of inputs with the number of degrees of freedom when the system is written as $\mathbf{x} = [q, \dot{q}]^\top$. A system can be controllable and underactuated at the same time.

Underactuation. $(\mathbf{A}_3, \mathbf{B}_3)$ has $x = [q, \dot{q}]^\top$ with a scalar q and one input acting directly on $\ddot{q}$: it is fully actuated. $(\mathbf{A}_4, \mathbf{B}_4)$ has $q = [x, \theta]^\top$ (two degrees of freedom) but only one input acting on $\ddot{x}$; the angle θ has no actuator, so the system is *underactuated*. This shows that controllability does not imply full actuation: $(\mathbf{A}_4, \mathbf{B}_4)$ is controllable but underactuated.

9 Section 3.8: Exercise 3.4

9.1 Model and linearisation

For a pendulum with $m = l = 1$, gravity $g = 9.81$ and viscous damping coefficient b,

$$\dot{\theta} = \omega, \qquad \dot{\omega} = -g \sin\theta - b\omega, \tag{35}$$

which corresponds to $ml^2\ddot{\theta} + b\dot{\theta} + mgl\sin\theta = 0$. The equilibria are $\mathbf{x}^\star = [0, 0]^\top$ (down) and $[\pi, 0]^\top$ (up). The Jacobian is

$$A(\theta^\star) = \begin{bmatrix} 0 & 1 \\ -g\cos\theta^\star & -b \end{bmatrix}. \tag{36}$$

Table 5: Eigenvalues of the Jacobian.

Condition	$\lambda_{1,2}$	$\operatorname{sign}(\operatorname{Re}\lambda_{1,2})$
$b = 0$, $\theta^\star = 0$	$\pm 3.1321j$	$0, 0$
$b = 0$, $\theta^\star = \pi$	± 3.1321	$+, -$
$b = 1$, $\theta^\star = 0$	$-0.5 \pm 3.0919j$	$-, -$
$b = 1$, $\theta^\star = \pi$	$2.6718, -3.6718$	$+, -$

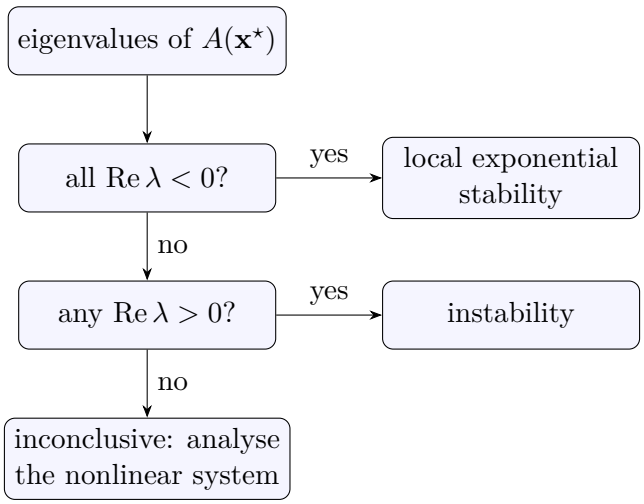


Figure 25: Decision diagram for the linearisation test. The first two branches are the standard Lyapunov indirect theorem. The third branch ($\operatorname{Re}\lambda = 0$) is the dangerous one: the linearisation alone cannot decide, and one must go back to the nonlinear system.

The three stability words in plain language

- **Stable i.s.L.** (in the sense of Lyapunov): if you start close enough, you stay close. You may oscillate forever.
- **Asymptotically stable**: you stay close *and* you eventually converge to the equilibrium.
- **Exponentially stable**: you converge at least as fast as e^{-ct} for some $c > 0$.

For the undamped pendulum at the bottom, the energy is conserved: the trajectory is a closed orbit. It is stable i.s.L. but not asymptotically stable, so it is certainly not exponentially stable. For the damped pendulum, the energy decreases and the trajectory spirals into the bottom: all three properties hold (locally). For the upright pendulum, a small perturbation makes the pendulum fall: the equilibrium is a saddle and is unstable.

Why $\operatorname{Re}\lambda = 0$ needs care

At the undamped bottom, the eigenvalues are $\pm 3.13j$: the real parts are zero. The linearised system is a centre, but the linearisation theorem is inconclusive. In this particular case the nonlinear system is also stable, but that is a conclusion from the energy function, not from the eigenvalues. A different nonlinear term with the same linearisation could be unstable. This is why the exercise asks you to draw the phase portrait and to use the nonlinear energy, not only the Jacobian.

9.2 Stability table

Table 6: Stability of the nonlinear pendulum and its linearisation. The nonlinear entries are local properties; the exponential entry for the damped hanging pendulum follows from the negative real parts of the linearisation.

System	sign $\operatorname{Re}\lambda_1$	sign $\operatorname{Re}\lambda_2$	Stable i.s.L.	Asymp. stable	Exp. stable
Nonlinear, $b = 0$, $(0, 0)$	N/A	N/A	Yes	No	No
Nonlinear, $b = 0$, $(\pi, 0)$	N/A	N/A	No	No	No
Linearization, $b = 0$, $(0, 0)$	0	0	Yes	No	No
Linearization, $b = 0$, $(\pi, 0)$	$+$	$-$	No	No	No
Nonlinear, $b = 1$, $(0, 0)$	N/A	N/A	Yes	Yes	Yes
Nonlinear, $b = 1$, $(\pi, 0)$	N/A	N/A	No	No	No
Linearization, $b = 1$, $(0, 0)$	$-$	$-$	Yes	Yes	Yes
Linearization, $b = 1$, $(\pi, 0)$	$+$	$-$	No	No	No

9.3 Discussion

1. For $b = 0$ at $(0, 0)$ the nonlinear system has closed orbits: it is stable i.s.L. but not asymptotically or exponentially stable. The linearisation is a centre; when $\operatorname{Re}\lambda = 0$ the linearisation theorem is inconclusive, and the nonlinear analysis (conserved energy) is required.

2. At $(\pi, 0)$ both the nonlinear system and its linearisation are unstable. The phase portrait is a saddle; the stable manifold is a measure-zero set, so almost every perturbation falls.

3. With $b = 1$ at $(0, 0)$ the eigenvalues have negative real parts and the nonlinear equilibrium is locally exponentially stable; the phase portrait spirals into the origin.

4. With $b = 1$ at $(\pi, 0)$ one eigenvalue remains positive; damping does not stabilise the inverted pendulum. Feedback is required.

5. Linearisation determines local exponential stability when all eigenvalues have negative real parts, and instability when at least one has a positive real part. It says nothing about global behaviour, finite escape times or the size of the basin of attraction.

10 Conclusions

1. **Underactuation vs. controllability.** The acrobot and the cart-pole are underactuated, yet their linearisations are completely controllable. Controllability is a rank property; the singular values and condition numbers reveal how hard the weakly controllable directions are to drive.

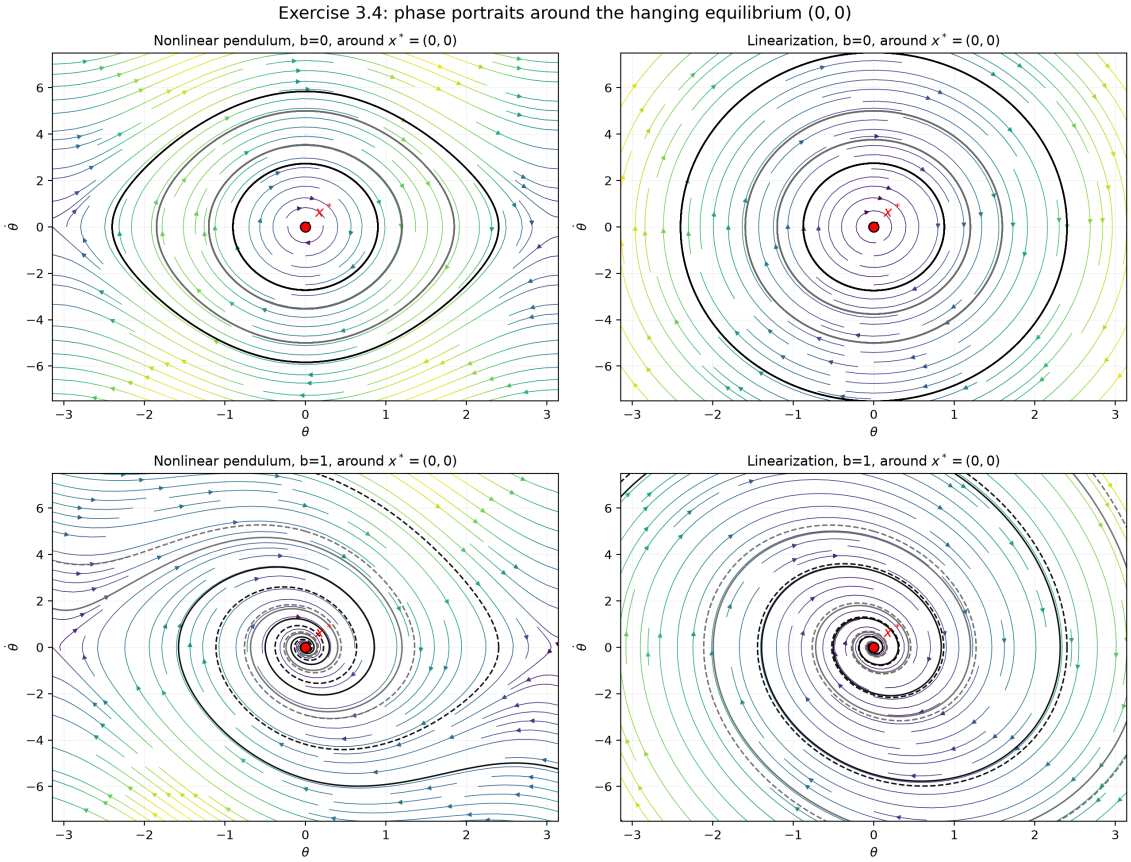


Figure 26: Phase portraits around the hanging equilibrium $(0, 0)$: nonlinear (left) and linearised (right), for $b = 0$ (top) and $b = 1$ (bottom).

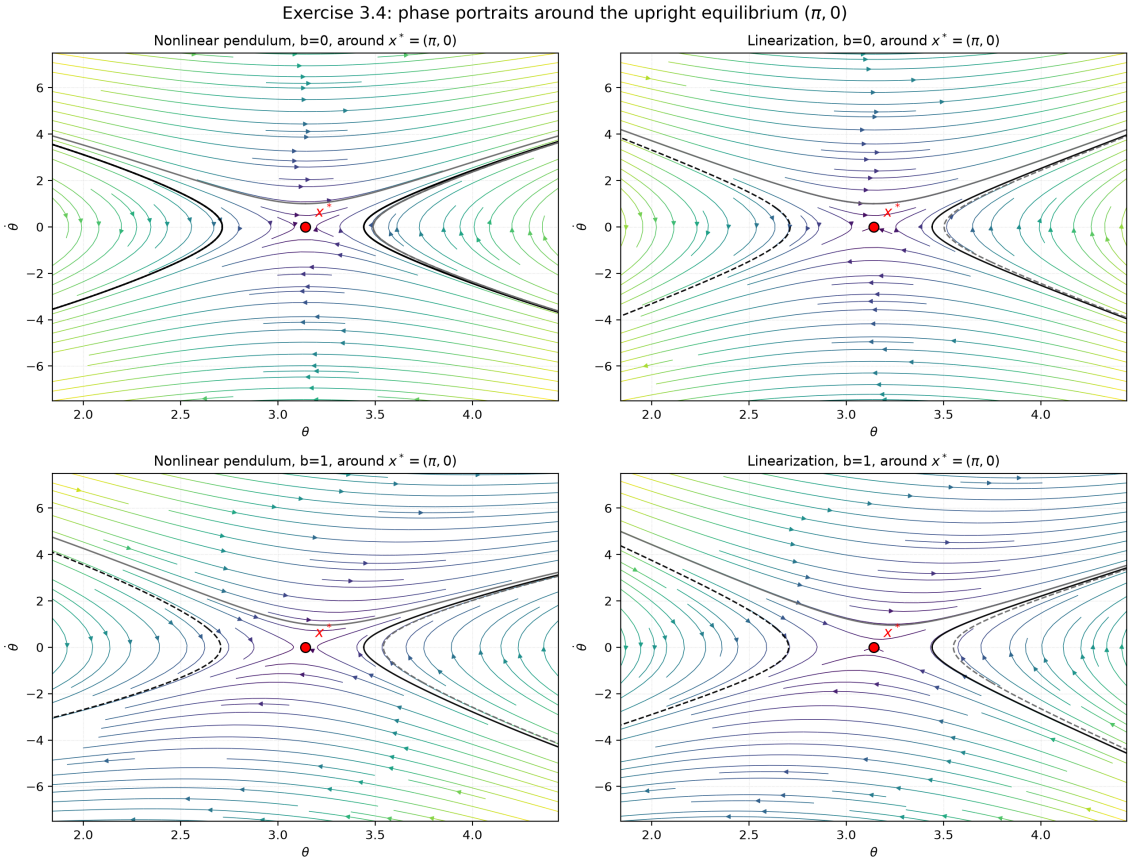


Figure 27: Phase portraits around the upright equilibrium $(\pi, 0)$: nonlinear (left) and linearised (right), for $b = 0$ (top) and $b = 1$ (bottom).

2. **Local nature of linear control.** LQR stabilises the linearisation and therefore gives local exponential stability. The nonlinear simulations show finite basins of attraction. Input saturation shrinks these basins substantially, especially for the acrobot with its m_2 sensitivity.

3. **Non-minimum phase.** Both the acrobot and the cart-pole have right-half-plane zeros. State feedback cannot remove them; the closed loop exhibits undershoot, which is visible in the successful trajectories and explains part of the difficulty under saturation.

4. **Quadrotor LQR.** LQR works out of the box around a hover equilibrium with non-zero thrust. The Q/R trade-off and the absence of integral action (steady-state wind offset) are clearly visible in the planar study; the 3D model recovers from large attitude perturbations.

5. **Section 3.8.** The cart-pole linearisation is valid only near the upright point; the URDF single/double pendula were built and balanced (autograder 13/13 and 12/12); the discrete double integrator is reachable in two steps; only (A_3, B_3) and (A_4, B_4) of Exercise 3.3 are controllable, and (A_4, B_4) is controllable but underactuated; the stability table of Exercise 3.4 separates the marginal, stable and unstable cases and highlights the limits of linearisation at zero real part.

11 Code and Reproduction

The complete code is provided in the repository. The main entry points are:

Topic	Script / notebook
Example 3.1 Acrobot	`exp1_analysis/exp1_analysis.py`, `Exp3'1-acrobot.ipynb`
Example 3.2 Cart-pole	`exp3_2_analysis/cp_analysis.py`, `Exp3'2-cartpole.ipynb`
Example 3.5 LQR comparison	`exp3_5_analysis/lqr_comparison.py`, `Exp3'5-LQR-analysis.ipynb`
Example 3.6 Quadrotor	`exp3_6_analysis/quadrotor_analysis.py`
	`Exp3'6-planar-quadrotor.ipynb`, `Exp3'6-quadrotor.ipynb`
Exercise 3.1	`cartpole_balancing.ipynb`, `ex3_1_balancing_analysis.py`
Exercise 3.2	`cartpoles_urdf.ipynb`, `ex3_2_urdf_lqr_analysis.py`
Exercise 3.3	`ex3_3_controllability.py`
Exercise 3.4	`ex3_4_stability_linearization.py`

Commands (from the corresponding directories):

```
/Users/leo/venvs/drake/bin/python exp1_analysis/exp1_analysis.py
/Users/leo/venvs/drake/bin/python exp3_2_analysis/cp_analysis.py
/Users/leo/venvs/drake/bin/python exp3_5_analysis/lqr_comparison.py
/Users/leo/venvs/drake/bin/python exp3_6_analysis/quadrotor_analysis.py
/Users/leo/venvs/drake/bin/python exercises_analysis/ex3_3_controllability.py
/Users/leo/venvs/drake/bin/python exercises_analysis/ex3_4_stability_linearization.py
```

The notebooks with autograders are executed headlessly with

```
COLAB_TESTING=1 /Users/leo/venvs/drake/bin/jupyter nbconvert \
  --to notebook --execute --inplace --ExecutePreprocessor.timeout=180 \
  cartpoles_urdf.ipynb
```

`COLAB_TESTING=1` makes `running_as_notebook` false so that `ModelVisualizer.Run(loop_once=True)` does not block.

A Key Code Snippets

A.1 Cart-pole linearisation and LQR

```
def f(x, u):
    c, s = np.cos(x[1]), np.sin(x[1])
    g = 9.81
    return np.array([
        x[2], x[3],
        (u[0] + s * (x[3]**2 + g * c)) / (1 + s**2),
        (-u[0] * c - x[3]**2 * c * s - 2 * g * s) / (1 + s**2)])

A = np.array([[0,0,1,0], [0,0,0,1],
              [0,9.81,0,0], [0,19.62,0,0]])
B = np.array([[0],[0],[1],[1]])
P = solve_continuous_are(A, B, Q, R)
K = np.linalg.solve(R, B.T @ P)
u = -K @ (x - x_star)
```

A.2 URDF pendulum link and joint

```
<link name="pendulum0">
  <inertial>
    <origin xyz="0 0 -1" />
    <mass value="1" />
    <inertia ixx="0" ixy="0" ixz="0" iyy="0" iyz="0" izz="0" />
  </inertial>
  <visual>
    <origin xyz="0 0 -1" />
    <geometry><sphere radius="0.05" /></geometry>
  </visual>
  <visual>
    <origin xyz="0 0 -0.5" />
    <geometry><cylinder length="1" radius="0.01" /></geometry>
  </visual>
</link>
<joint name="theta0" type="continuous">
  <parent link="base" />
  <child link="pendulum0" />
  <origin xyz="0 0 0" />
  <axis xyz="0 -1 0" />
</joint>
```

A.3 Controllability and two-step reachability

```
def controllability_matrix(A, B):
    n = A.shape[0]
    cols = [B]
    for _ in range(1, n):
        cols.append(A @ cols[-1])
    return np.hstack(cols)
```

```
def two_step_control(start, goal):
    i1, j1 = start
    i2, j2 = goal
    u0 = i2 - i1 - 2 * j1
    u1 = j2 - j1 - u0
    return (u0, u1)
```

A.4 Quadrotor LQR

```
context = plant.CreateDefaultContext()
context.SetContinuousState(x_star)
u_hover = np.array([mass * gravity / 4.0] * 4)
plant.get_input_port(0).FixValue(context, u_hover)
Q = np.diag(np.concatenate(([10.0] * 6, [1.0] * 6)))
R = np.eye(4)
lqr = LinearQuadraticRegulator(plant, context, Q, R)
builder.Connect(lqr.get_output_port(0), plant.get_input_port(0))
builder.Connect(plant.get_output_port(0), lqr.get_input_port(0))
```